

Customer Churn Prediction System

Linear Regression • Random Forest • XGBoost

Live demo: customer-churn-prediction-on.streamlit.app

PRESENTED BY

SUBASH ACHARYA D

Roll No. 24CSR__

THANNASI PRABU R

Roll No. 24CSR__

VIKRAM S

Roll No. 24CSR__

Problem Statement

Why churn prediction matters

Challenges

- Annual losses exceed ₹10,000 crore across telecom, banking and OTT due to churn.
- Rule-based retention systems miss complex behavioural patterns (usage shifts, payment irregularities, service interactions).
- Generic global datasets ignore localised drivers such as UPI usage trends and regional behaviour.
- Businesses react too late — churn signals are not detected early enough.
- No production-ready, interpretable models at 85–92% accuracy.

Proposed Solution

- Train Linear Regression, Random Forest and XGBoost on Telco customer churn data.
- Detect churn early using churn probability + risk levels for every customer.
- Explain every prediction (local drivers + global importance).
- Serve insights through a Flask web app and a deployed Streamlit dashboard.
- Deliver reproducible, interpretable ML with honest, test-set-validated metrics.

06

Online Deployment

Host the app publicly on Streamlit
Community Cloud from the GitHub repo.

Dataset Information

Telco Customer Churn — IBM sample dataset

7,032

Records after cleaning

20

Features

26.6%

Churn rate

1,407

Hold-out test customers

Core dataset fields

- Demographics: gender, SeniorCitizen, Partner, Dependents
- Usage: tenure, PhoneService, MultipleLines, InternetService
- Services: OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, Streaming
- Contract & billing: Contract, PaperlessBilling, PaymentMethod, Monthly / Total Charges

Preprocessing in `src/preprocess.py`

- Drop customerID; convert TotalCharges to numeric
- Map binary fields (gender, Partner, Dependents, services) to 0/1
- One-hot encode category fields; standardise numeric fields
- Engineer avg_charge_per_month and tenure_log
- Target: Churn → Yes/No (26.6% positive class)

Model Choosing

Linear Regression

Baseline

- Continuous score thresholded at 0.5
- Strong ROC-AUC 0.837
- Coefficient-weighted local explanations
- 79.8% accuracy on the test set

Random Forest

Retention champion

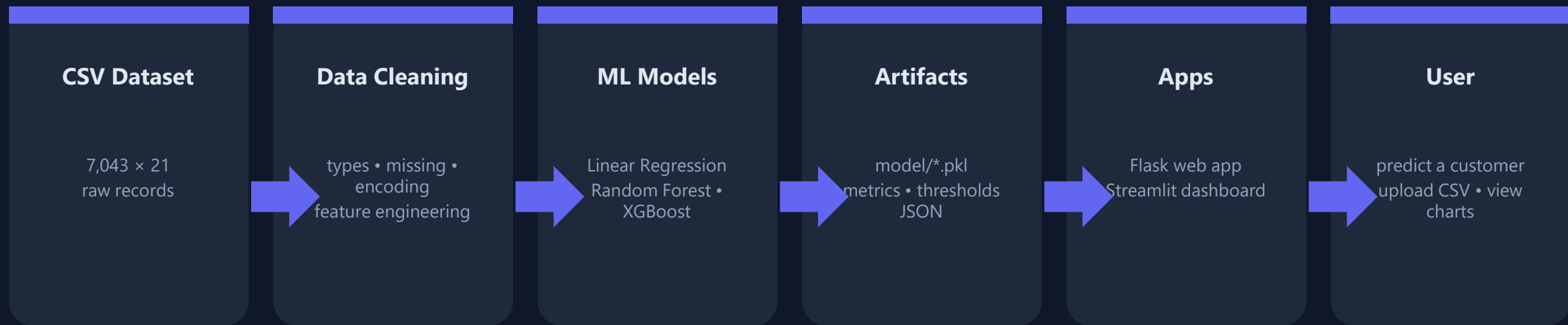
- 400 trees, max depth 12, balanced classes
- Best churn recall — finds 76.2% of churners
- Probability calibrated with threshold 0.70
- Recommended for retention teams

XGBoost

Gradient boosting

- 500 rounds, learning rate 0.05
- Mild imbalance weight 1.4
- Clean probability estimates
- 77.2% accuracy, AUC 0.817

System Architecture



User selects a customer / uploads data → all three models score it → interpretations and visual analytics

Workflow

1

Load Data

Read Telco_Customer_Churn.csv

2

Clean Data

Convert types • drop NA • encode • engineer features

3

Train Models

Linear Regression • Random Forest • XGBoost

4

Save Artifacts

Full pipelines, metrics, thresholds via joblib/JSON

Inputs

Customer profile / CSV upload — predictions, probabilities, risk levels

Outputs

Verdicts + churn probability, interpretation, CSV downloads, live charts

Application Modules

Flask Web App (`app.py`)

- Model dashboard — metrics & confusion matrices
- Single prediction — 3-model verdicts + explanation
- Batch CSV upload with ensemble verdict
- JSON API — POST `/api/predict`

Streamlit App (`streamlit_app.py`)

- Model dashboard — KPIs, bars, confusion matrices
- Predict a customer — form + live explanation
- Batch upload — downloadable results
- About — methodology and honest performance notes

Both apps share the same trained pipelines and preprocessing — one codebase, two front-ends.

Model Performance & Validation

80/20 stratified train-test split • 1,407 test customers • all metrics test-set only

79.8%

Linear Regression accuracy

76.2%

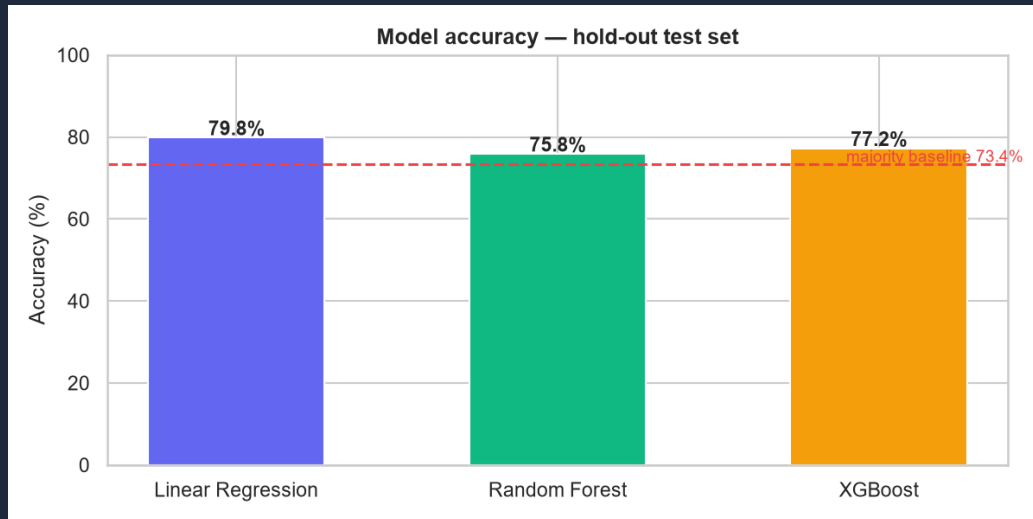
Random Forest churn recall

0.837

Best ROC-AUC (Linear Regression)

73.4%

Majority-class baseline



Validation notes

- Stratified split keeps churn rate constant
- Accuracy-optimised decision thresholds per model
- Honest note: 85–92% is aspirational on this dataset — 75–80% is the realistic state of the art
- Random Forest finds 76% of future churners

Conclusion & Future Scope

Project Outcome

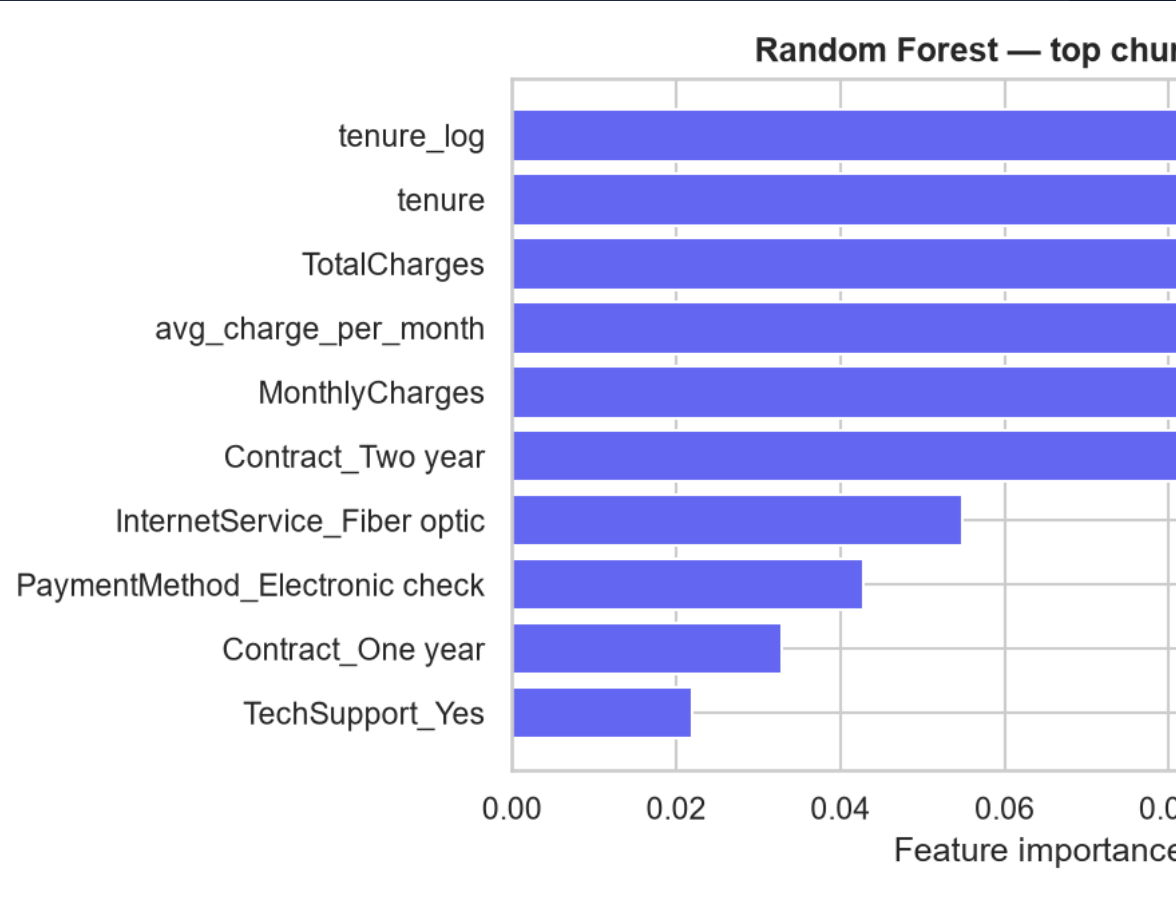
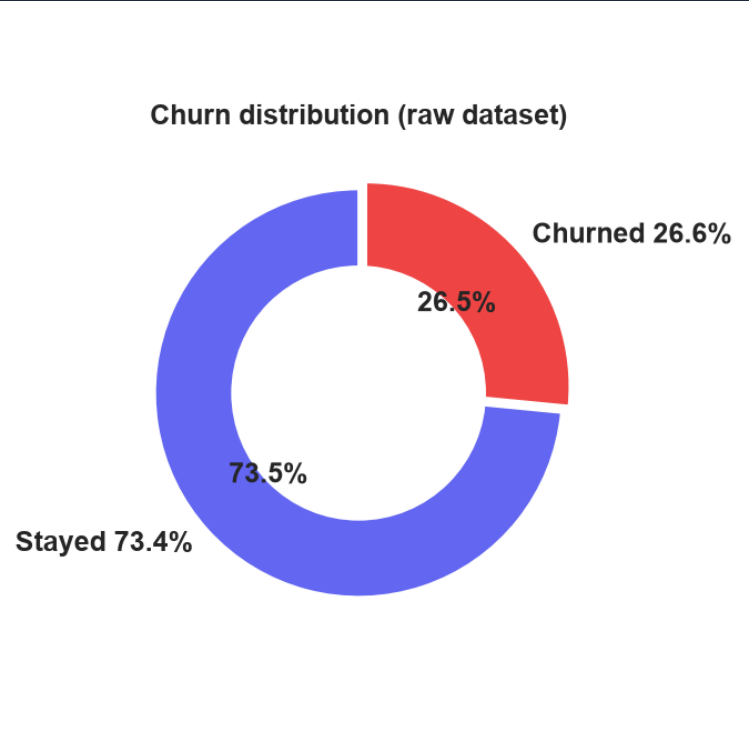
- End-to-end churn system: clean → train → evaluate → deploy
- Three interpretable models with honest, test-set metrics
- Flask + Streamlit front-ends and a JSON API
- Publicly accessible Streamlit dashboard on the cloud
- Batch scoring with downloadable reports

Future Improvements

- Localised data: UPI trends, migration patterns, regional behaviour
- SHAP / LIME explanations for every prediction
- Time-series and uplift models for retention campaigns
- Automated alerts for high-risk customers
- Model monitoring as new data arrives

Output — Dashboard & Predictions

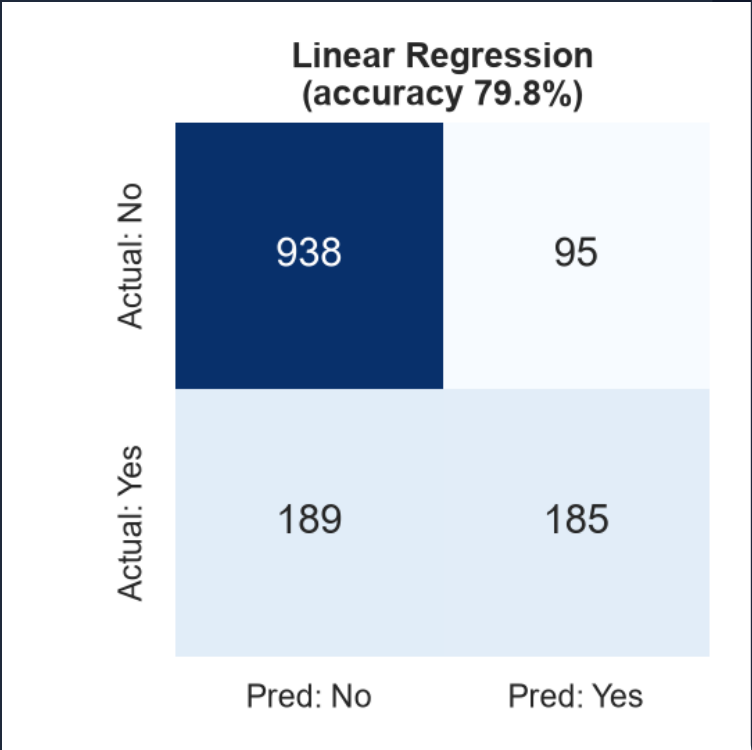
Live app: <https://customer-churn-prediction-on.streamlit.app>



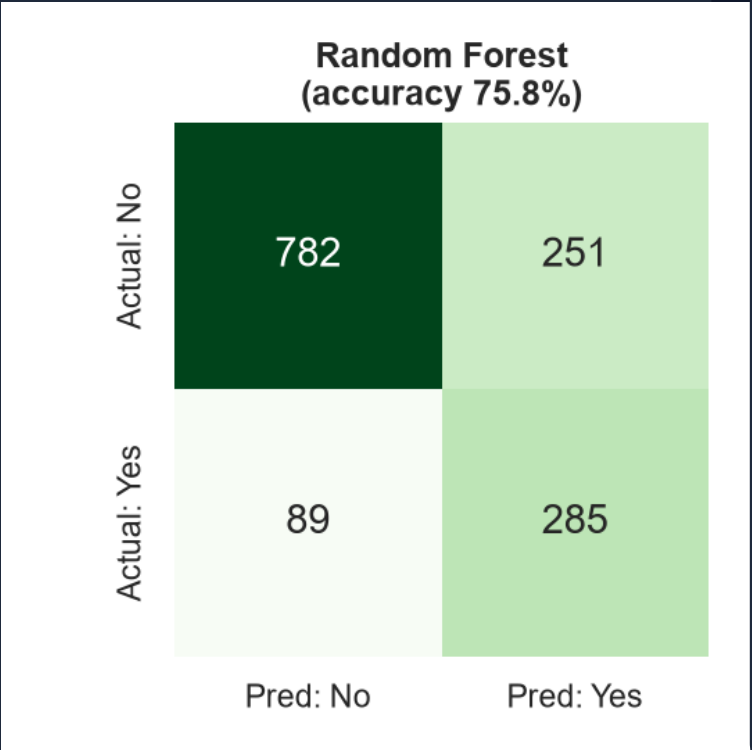
Every customer gets a churn probability, risk level, consensus verdict and an explanation of the deciding features.

Output — Confusion Matrices (hold-out test set)

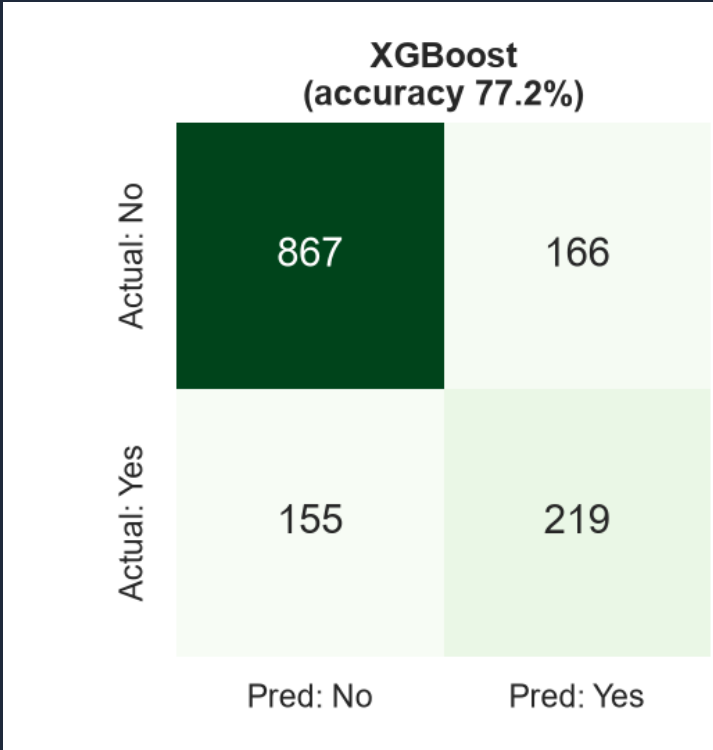
Linear Regression



Random Forest



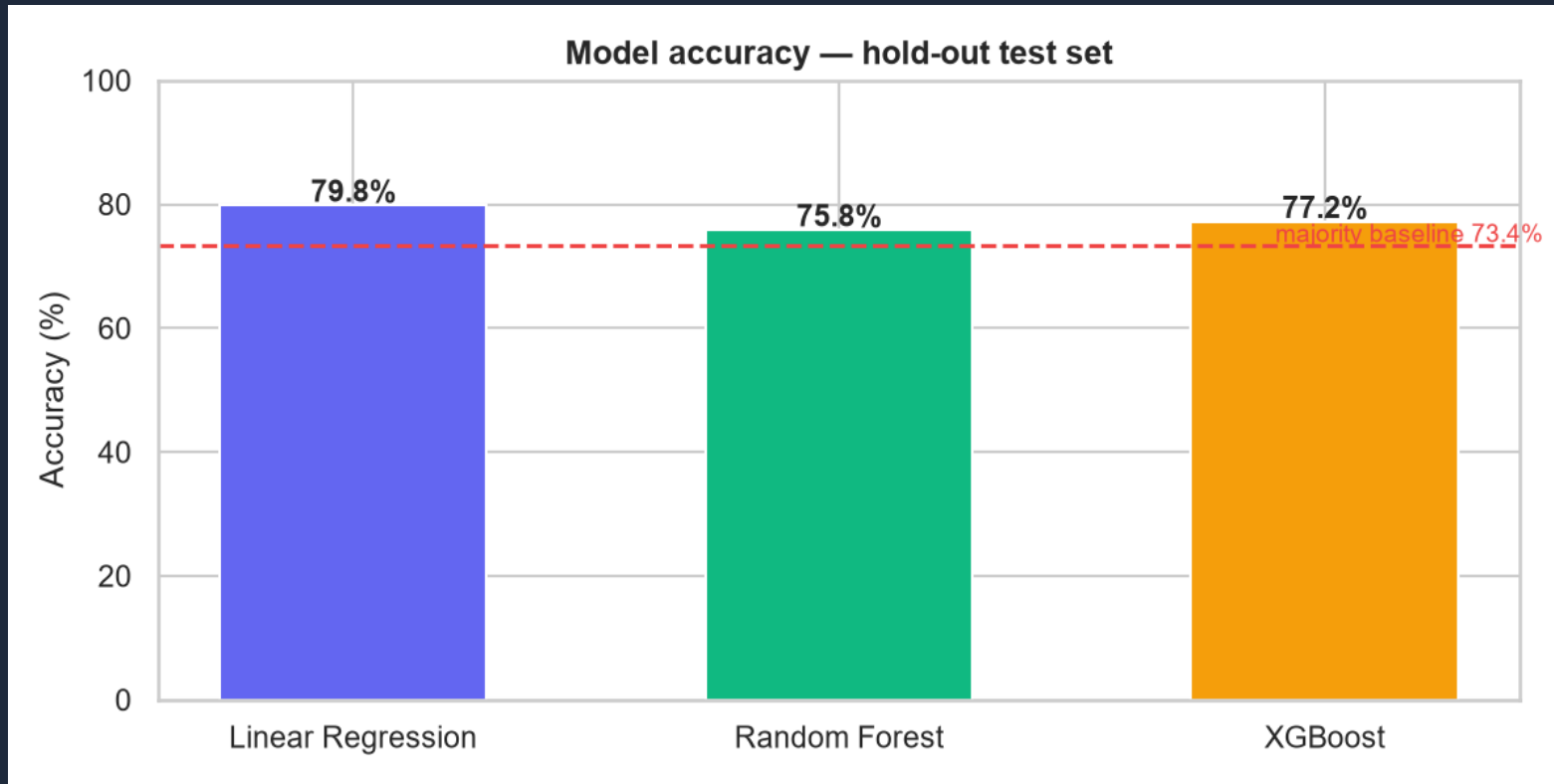
XGBoost



Random Forest trades a little accuracy for far higher churn recall — the right trade for retention teams.

Output — Model Comparison Dashboard

Full interactive analytics on the deployed Streamlit app



Accuracy, precision, recall, F1, ROC-AUC and confusion matrices for all three models, side by side.